SE 2228 Analysis & Design of Algorithms, Yaşar University, 2025-2026 Fall

Lab 02: The Tutorial's Recurrences in C

Mert Hazar Hasgül, mert.hasgul@yasar.edu.tr

The five recurrences of *A Short Tutorial on Recurrence Relations* (`Tutorial_on_recurrence_relations.pdf`, in this folder, by a separate author), each typed as C and run next to the closed form the tutorial derives. The algorithms in sections 4 and 5 come from the longer text version of the same tutorial. Code is in `tutorial-code/`. Compile any file with:

```
TERMINAL
gcc -std=c11 -Wall -Wextra -pedantic file.c -o file
```

Contents

1. g(n) = g(n−1) + 2n − 1, g(0) = 0
2. Fibonacci: f(n) = f(n−1) + f(n−2), f(0) = f(1) = 1
3. Two variables: T(m, n) = 2T(m/2, n/2) + mn
4. Test: T(n) = T(n−1) + O(n)
5. Parallel-Product: T(n) = T(n/2) + O(n/2)

1. g(n) = g(n−1) + 2n − 1, g(0) = 0

DERIVATION

```
g(n) = g(n−1) + 2n − 1,   g(0) = 0
     = 0 + 2 + 4 + ... + 2n − n                    ← unfolded n times, g(0) = 0
     = 2·n(n+1)/2 − n = n²
```

tutorial-code/1-g.c

```c
/* The tutorial's first recurrence: g(n) = g(n-1) + 2n - 1, g(0) = 0.  Closed form: n^2. */
/* Mert Hazar HASGUL - mert.hasgul@yasar.edu.tr */
#include <stdio.h>

long g(int n) {
    if (0 == n) {
        return 0;                       /* base case */
    }
    return g(n - 1) + 2L * n - 1;       /* recurrence */
}

int main(void) {
    static const int ns[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 100, 1000};
    size_t t;
    int n;

    printf("   n     g(n)      n^2\n");
    for (t = 0; t < sizeof ns / sizeof ns[0]; t++) {
        n = ns[t];
        printf("%4d %8ld %8ld\n", n, g(n), (long)n * n);
    }
    return 0;
}
```

OUTPUT

```
   n     g(n)      n^2
   0        0        0
   1        1        1
   2        4        4
   3        9        9
   4       16       16
   5       25       25
   6       36       36
   7       49       49
   8       64       64
   9       81       81
  10      100      100
 100    10000    10000
1000  1000000  1000000
```

- The `if` is the base case, the `return` is the recurrence. g(n) and n^2 agree on every row.

2. Fibonacci: f(n) = f(n−1) + f(n−2), f(0) = f(1) = 1

ALGORITHM

```
f(n) = f(n−1) + f(n−2),   f(0) = f(1) = 1              → 1, 1, 2, 3, 5, 8, 13, ...

Algorithm Fib(n):                                       → O(n)
    f[0] := f[1] := 1
    for i := 2 to n do  f[i] := f[i−1] + f[i−2]          ← following the recurrence

Algorithm Fib-Rec(n):
    if n = 0 or 1 then return 1
    else return Fib-Rec(n−1) + Fib-Rec(n−2)
    → T(n) = T(n−1) + T(n−2),  T(0) = T(1) = 1:  exponential
```

tutorial-code/2-fibonacci.c

```c
/* The tutorial's Fibonacci recurrence f(n) = f(n-1) + f(n-2), f(0) = f(1) = 1,
   as its two algorithms: Fib(n) (dynamic programming, O(n)) and Fib-Rec(n) (exponential). */
/* Mert Hazar HASGUL - mert.hasgul@yasar.edu.tr */
#include <stdio.h>

static long calls;                      /* number of Fib-Rec calls */

/* Algorithm Fib(n): fill f[0..n] following the recurrence. */
long fib(int n) {
    long f[64];
    int i;

    f[0] = 1;
    f[1] = 1;
    for (i = 2; i <= n; i++) {
        f[i] = f[i - 1] + f[i - 2];     /* following the recurrence relation */
    }
    return f[n];
}

/* Algorithm Fib-Rec(n): the recurrence itself. T(n) = T(n-1) + T(n-2) calls. */
long fib_rec(int n) {
    calls++;
    if (n <= 1) {
        return 1;                       /* f(0) = f(1) = 1 */
    }
    return fib_rec(n - 1) + fib_rec(n - 2);
}

int main(void) {
    static const int ns[] = {0, 1, 2, 3, 4, 5, 10, 20, 30};
    size_t t;
    int n;
    long f;

    printf("   n   Fib(n)  Fib-Rec(n)  Fib-Rec calls  2f(n)-1\n");
    for (t = 0; t < sizeof ns / sizeof ns[0]; t++) {
        n = ns[t];
        calls = 0;
        f = fib_rec(n);
        printf("%4d %8ld %11ld %14ld %8ld\n", n, fib(n), f, calls, 2 * f - 1);
    }
    return 0;
}
```

```
OUTPUT
   n   Fib(n)  Fib-Rec(n)  Fib-Rec calls  2f(n)-1
   0        1           1              1        1
   1        1           1              1        1
   2        2           2              3        3
   3        3           3              5        5
   4        5           5              9        9
   5        8           8             15       15
  10       89          89            177      177
  20    10946       10946          21891    21891
  30  1346269     1346269        2692537  2692537
```

- Both algorithms return the same f(n). For n = 30, `Fib` does 29 additions; `Fib-Rec` makes 2.7 million calls.
- The tutorial's T(n) = T(n−1) + T(n−2), T(0) = T(1) = 1 is f(n) itself. The call count is 2f(n) − 1: exponential, about 1.618^n.

3. Two variables: T(m, n) = 2T(m/2, n/2) + mn

DERIVATION

$$
\begin{aligned}
T(m,n) &= 2T(m/2,\ n/2) + mn \quad (m,\ n > 1), \quad T(1,\ n) = n, \quad T(m,\ 1) = m \\
&= 2^k T(1,\ n/2^k) + mn/2^{k-1} + \ldots + mn/2 + mn && \leftarrow\ m \le n,\ k = \log_2 m \\
&= mn \cdot (2 - 1/2^k) = 2mn - n = \theta(mn) && \leftarrow\ 2^k = m
\end{aligned}
$$

tutorial-code/3-two-variables.c

```c
/* The tutorial's two-variable recurrence:
     T(m,n) = 2T(m/2,n/2) + mn  (m > 1, n > 1),   T(1,n) = n,   T(m,1) = m.
   Closed form for m <= n and k = log2(m):  mn(2 - 1/2^k) = 2mn - n. */
/* Mert Hazar HASGUL - mert.hasgul@yasar.edu.tr */
#include <stdio.h>

long T(long m, long n) {
    if (1 == m) {
        return n;                       /* base case m = 1 */
    }
    if (1 == n) {
        return m;                       /* base case n = 1 */
    }
    return 2 * T(m / 2, n / 2) + m * n; /* recurrence */
}

int main(void) {
    static const long mn[][2] = {{1, 1}, {2, 2}, {2, 8}, {4, 4},
                                 {4, 16}, {8, 8}, {8, 64}, {64, 1024}};
    size_t t;
    long m;
    long n;
    long p;                             /* p = 2^k = m */

    printf("     m      n    T(m,n)  mn(2-1/2^k)   2mn-n\n");
    for (t = 0; t < sizeof mn / sizeof mn[0]; t++) {
        m = mn[t][0];
        n = mn[t][1];
        for (p = 1; p < m; p *= 2) {
        }
        printf("%6ld %6ld %9ld %12ld %7ld\n",
               m, n, T(m, n), 2 * m * n - m * n / p, 2 * m * n - n);
    }
    return 0;
}
```

OUTPUT

```
     m      n    T(m,n)  mn(2-1/2^k)   2mn-n
     1      1         1            1       1
     2      2         6            6       6
     2      8        24           24      24
     4      4        28           28      28
     4     16       112          112     112
     8      8       120          120     120
     8     64       960          960     960
    64   1024    130048       130048  130048
```

- Every row has m ≤ n and m = 2^k, as the tutorial assumes. mn(2 − 1/2^k) and 2mn − n are the same number because 2^k = m.

4. Test: T(n) = T(n−1) + O(n)

ALGORITHM

```
Algorithm Test(A[1..n], B[1..n], C[1..n]):
    if n = 0 then return
    for i := 1 to n do  C[i] := A[i] * B[i]              ← n multiplications: O(n)
    Test(A[2..n], B[2..n], C[2..n])                      ← T(n−1)

T(n) = T(n−1) + O(n),  T(1) = 1
     = T(1) + O(2) + ... + O(n−1) + O(n) = O(1 + 2 + ... + n) = O(n²)
```

tutorial-code/4-test.c

```c
/* The tutorial's Algorithm Test(A[1..n], B[1..n], C[1..n]):
     if n = 0 then return;
     for i := 1 to n do C[i] := A[i] * B[i];
     call Test(A[2..n], B[2..n], C[2..n]);
   T(n) = T(n-1) + O(n), T(1) = 1  ->  O(n^2).   Counting multiplications: T(n) = T(n-1) + n. */
/* Mert Hazar HASGUL - mert.hasgul@yasar.edu.tr */
#include <stdio.h>

static long mults;                      /* multiplications done by test() */

void test(const int a[], const int b[], int c[], int n) {
    int i;

    if (0 == n) {
        return;                         /* base case */
    }
    for (i = 0; i < n; i++) {
        c[i] = a[i] * b[i];             /* n multiplications: the O(n) part */
        mults++;
    }
    test(a + 1, b + 1, c + 1, n - 1);   /* T(n-1): the same on A[2..n], B[2..n], C[2..n] */
}

int main(void) {
    static const int ns[] = {1, 2, 3, 4, 5, 10, 100, 1000};
    static int a[1000];
    static int b[1000];
    static int c[1000];
    size_t t;
    int n;
    int i;

    for (i = 0; i < 1000; i++) {
        a[i] = i + 1;
        b[i] = 2;
    }
    printf("   n   multiplications   n(n+1)/2\n");
    for (t = 0; t < sizeof ns / sizeof ns[0]; t++) {
        n = ns[t];
        mults = 0;
        test(a, b, c, n);
        printf("%4d %17ld %10ld\n", n, mults, (long)n * (n + 1) / 2);
    }
    return 0;
}
```

```
OUTPUT
   n   multiplications   n(n+1)/2
   1                 1          1
   2                 3          3
   3                 6          6
   4                10         10
   5                15         15
  10                55         55
 100              5050       5050
1000            500500     500500
```

- Multiplications = n + (n−1) + ... + 1 = n(n+1)/2: the tutorial's $O(n^2)$, exactly.
- The text version writes the loop body as C[1] := A[1] * B[i]; it is C[i] := A[i] * B[i] here. The count is the same.

5. Parallel-Product: T(n) = T(n/2) + O(n/2)

ALGORITHM

```
Algorithm Parallel-Product(A[1..n]):                    (n = 2^k)
    if n = 1 then return
    for i := 1 to n/2 do  A[i] := A[i] * A[i + n/2]     ← n/2 multiplications: O(n/2)
    Parallel-Product(A[1..n/2])                         ← T(n/2)

T(n) = T(n/2) + O(n/2),  T(1) = 1
     = T(1) + O(n/2^k) + ... + O(n/4) + O(n/2) = 1 + O(n·(1/2^k + ... + 1/4 + 1/2)) = O(n)
```

tutorial-code/5-parallel-product.c

```c
/* The tutorial's Algorithm Parallel-Product(A[1..n]), n a power of 2:
     if n = 1 then return;
     for i := 1 to n/2 do A[i] := A[i] * A[i + n/2];
     call Parallel-Product(A[1..n/2]);
   T(n) = T(n/2) + O(n/2), T(1) = 1  ->  O(n).   Counting multiplications: T(n) = T(n/2) + n/2. */
/* Mert Hazar HASGUL - mert.hasgul@yasar.edu.tr */
#include <stdio.h>

static long mults;                      /* multiplications done by parallel_product() */

void parallel_product(long a[], int n) {    /* afterwards a[0] = product of all n elements */
    int i;

    if (1 == n) {
        return;                         /* base case */
    }
    for (i = 0; i < n / 2; i++) {
        a[i] = a[i] * a[i + n / 2];     /* n/2 multiplications: the O(n/2) part */
        mults++;
    }
    parallel_product(a, n / 2);         /* T(n/2): the same on A[1..n/2] */
}

int main(void) {
    static const int ns[] = {1, 2, 4, 8, 16, 64, 256, 1024};
    static long a[1024];
    size_t t;
    int n;
    int i;

    printf("   n   multiplications    n-1   a[0]\n");
    for (t = 0; t < sizeof ns / sizeof ns[0]; t++) {
        n = ns[t];
        for (i = 0; i < n; i++) {
            a[i] = 1;                   /* all ones ...                          */
        }
        a[n - 1] = 3;                   /* ... except the last, so the product is 3 */
        mults = 0;
        parallel_product(a, n);
        printf("%4d %17ld %6d %6ld\n", n, mults, n - 1, a[0]);
    }
    return 0;
}
```

```
OUTPUT
   n   multiplications    n-1   a[0]
   1                 0      0      3
   2                 1      1      3
   4                 3      3      3
   8                 7      7      3
  16                15     15      3
  64                63     63      3
 256               255    255      3
1024              1023   1023      3
```

- Multiplications = n/2 + n/4 + ... + 1 = n − 1. The tutorial's T(1) = 1 also counts the last call, which makes its T(n) = n. Both are the O(n) of the tutorial.
- a[0] = 3 on every row: the product of all n elements, which is what the algorithm computes.